Simulacro Prueba de Desempeño .NET — FrameRent

FrameRent es una empresa que alquila equipos audiovisuales: cámaras, luces, micrófonos, trípodes y accesorios. Sus clientes son fotógrafos, productoras pequeñas y creadores de contenido que necesitan equipo por unos días.

Hoy todo se maneja con hojas de cálculo y mensajes de WhatsApp. Esto genera alquileres duplicados, equipos prometidos que no estaban disponibles y cobros calculados a mano con errores.

FrameRent quiere una plataforma donde sus clientes consulten el catálogo y soliciten alquileres, y donde el personal de la empresa gestione el inventario y controle todo el ciclo de cada alquiler, desde la solicitud hasta la devolución del equipo.

Objetivo

Construir una solución en .NET compuesta por una **API REST** y una **capa de presentación** que permita a FrameRent operar su negocio de alquiler.

Este simulacro evalúa tu capacidad para:

- Diseñar una API aplicando Clean Architecture.
- Modelar reglas de negocio reales en el dominio, más allá de un CRUD.
- Proteger la información con autenticación y autorización.
- Consumir esa lógica desde una aplicación cliente.

Todo lo que has aprendido en la ruta es válido y esperado en esta prueba.

Requerimientos funcionales

Usuarios y acceso

- Cualquier persona puede registrarse en la plataforma. Quien se registra lo hace siempre como **cliente**.
- FrameRent ya cuenta con una cuenta de su personal, con la que se debe poder ingresar desde el primer arranque de la aplicación.(recuerda dejar los datos del usuario en la documentacion)
- El personal de la empresa gestiona el catálogo y los alquileres de todos los clientes.
- Un cliente solo puede ver y gestionar **sus propios** alquileres. Nunca los de otro cliente.
- Cualquier visitante, aunque no haya iniciado sesión, puede consultar el catálogo y la disponibilidad de un equipo.

Equipos

De cada equipo se necesita conocer como mínimo: nombre, categoría, descripción, tarifa diaria, unidades totales que posee la empresa y si está activo.

- Solo el personal puede crear, editar o desactivar equipos.
- Un equipo **no se elimina**: se desactiva.
- Un equipo con alquileres aprobados o entregados no puede desactivarse.
- Un equipo inactivo no aparece en el catálogo y no se puede alquilar.

Alquileres

Un cliente solicita el alquiler de un equipo indicando la cantidad de unidades, la fecha de inicio y la fecha de fin.

1. **Fechas válidas.** La fecha de inicio no puede estar en el pasado y la fecha de fin debe ser posterior a la de inicio. Un alquiler dura como máximo **15 días**.
2. **Disponibilidad.** No se pueden alquilar más unidades de las disponibles en ese rango de fechas. Se descuentan las unidades de los alquileres aprobados o entregados que se crucen con esas fechas.
3. **Costo.** El costo total es días × tarifa diaria × cantidad. Lo calcula el sistema; cualquier valor de costo que envíe el cliente se ignora.
4. **Límite por cliente.** Un cliente no puede tener más de **3** alquileres pendientes o aprobados al mismo tiempo.
5. **Ciclo de vida.** Todo alquiler nace **Pendiente** y solo puede avanzar así:
    - Pendiente → Aprobado → Entregado → Devuelto
    - Pendiente → Rechazado
    - Pendiente o Aprobado → Cancelado

Cualquier otra transición debe rechazarse. Por ejemplo, no se puede entregar un alquiler que no fue aprobado. 6. **Aprobación.** Al aprobar, se vuelve a validar la disponibilidad: pudo cambiar desde que se hizo la solicitud. 7. **Cancelación.** El cliente puede cancelar su alquiler solo si está Pendiente o Aprobado y antes de la fecha de inicio. 8. **Gestión del personal.** El personal aprueba o rechaza solicitudes, registra la entrega del equipo y registra su devolución. 9. **Multa por retraso.** Si la devolución ocurre después de la fecha de fin, se cobra una multa: días de retraso × tarifa diaria × cantidad × **1.5**. La multa queda registrada en el alquiler.

Reporte

El personal necesita consultar los **equipos más alquilados**, ordenados de mayor a menor número de alquileres.

Endpoints de la API

La API debe exponer como mínimo los siguientes endpoints. Puedes agregar los que consideres necesarios. Los endpoints que no son públicos deben exigir un token JWT válido.

Autenticación

Método	Ruta	Acceso	Descripción
POST	`/api/v1/auth/register`	Público	Registra un nuevo cliente
POST	`/api/v1/auth/login`	Público	Inicia sesión y devuelve el token JWT

Equipos

Método	Ruta	Acceso	Descripción
GET	`/api/v1/equipment`	Público	Lista los equipos activos
GET	`/api/v1/equipment/{id}`	Público	Detalle de un equipo
GET	`/api/v1/equipment/{id}/availability?from=&to=`	Público	Unidades disponibles

Be a coder, change your world

			en un rango de fechas
POST	`/api/v1/equipment`	Personal	Crea un equipo
PUT	`/api/v1/equipment/{id}`	Personal	Actualiza un equipo
PATCH	`/api/v1/equipment/{id}/deactivate`	Personal	Desactiva un equipo

Alquileres

Método	Ruta	Acceso	Descripción
POST	`/api/v1/rentals`	Cliente	Solicita un alquiler
GET	`/api/v1/rentals/mine`	Cliente	Lista los alquileres del cliente autenticado
GET	`/api/v1/rentals/{id}`	Dueño del alquiler o personal	Detalle de un alquiler
PATCH	`/api/v1/rentals/{id}/cancel`	Dueño del alquiler	Cancela un alquiler
GET	`/api/v1/rentals?status=`	Personal	Lista todos los alquileres, con filtro opcional por estado
PATCH	`/api/v1/rentals/{id}/approve`	Personal	Aprueba una solicitud
PATCH	`/api/v1/rentals/{id}/reject`	Personal	Rechaza una solicitud
PATCH	`/api/v1/rentals/{id}/deliver`	Personal	Registra la entrega del equipo
PATCH	`/api/v1/rentals/{id}/return`	Personal	Registra la devolución y calcula la multa si aplica

Reportes

Método	Ruta	Acceso	Descripción
GET	`/api/v1/reports/top-equipment`	Personal	Equipos más alquilados

Las reglas de negocio se validan en el dominio y en la capa de aplicación, no en los controladores. Ante un error, la API debe responder con el código HTTP adecuado y un mensaje claro.

Capa de presentación

Además de la API, debes construir una aplicación para los clientes de FrameRent. Elige **una** de estas dos opciones:

Opción	Qué construir	Cómo se conecta
A. Web con ASP.NET Core	Una aplicación web dentro de la misma solución, como otra capa de presentación	Consume los **servicios de la capa de aplicación**, no los endpoints de la API
B. Cliente frontend	Una aplicación en la tecnología frontend que	Consume los endpoints de la API usando el token JWT

	prefieras(otro respositorio aparte)	

La aplicación debe cubrir el flujo del cliente:

- ☐ Registro e inicio de sesión.
- ☐ Consulta del catálogo y de la disponibilidad de un equipo.
- ☐ Solicitud de un alquiler.
- ☐ Consulta de sus alquileres y su estado.
- ☐ Cancelación de un alquiler.

Las operaciones del personal se demuestran directamente sobre la API, con Swagger, Scalar o Postman.

Requerimientos técnicos obligatorios

Si alguno de estos puntos no se cumple, la prueba se considera incompleta.

1. **Clean Architecture.** La solución debe separar como mínimo Domain, Application, Infrastructure y la capa de presentación (API y, si aplica, la web). Las dependencias apuntan hacia el dominio.
2. **Base de datos con Code First.** El modelo se define en código con Entity Framework Core y la base de datos se genera mediante migraciones.
3. **ASP.NET Core Identity** para la gestión de usuarios.
4. **JWT.** Los endpoints que no son públicos deben estar protegidos con token JWT.
5. **Autorización.** Cada operación debe estar restringida a quien corresponde según los requerimientos funcionales.
6. **Reglas de negocio** implementadas y validadas fuera de los controladores.
7. **Capa de presentación** según la opción elegida.
8. **Código en inglés:** clases, propiedades, métodos, carpetas y proyectos.
9. **README en inglés** que incluya:
    - Descripción del proyecto.
    - Tecnologías utilizadas.
    - Pasos para configurar y ejecutar la solución, incluida la base de datos.
    - Credenciales de la cuenta del personal.
    - Sección **Author** con tu nombre completo, clan, correo y documento de identidad.
10. **Repositorio en GitHub** con el historial de commits del desarrollo.

Opcionales

Hazlos **solo si terminaste todo lo obligatorio**. Un opcional no compensa ni subsana un requerimiento obligatorio que esté incompleto o mal implementado.

- ☐ Dockerizar el proyecto o los proyectos.
- ☐ Pruebas unitarias.
- ☐ Despliegue de la solución completa.

Entregables

- ☐ El proyecto completo, comprimido, según las indicaciones de tu Team Leader.
- ☐ El enlace al repositorio público en GitHub en caso de necesitarlo.
- ☐ El README en inglés en la raíz del repositorio.